PES UNIVERSITY – DEPT. OF CSE

Roopesh · SRN: PES1UG24AM393

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #26 – Smart Cities, Transport & Logistics

Peer-to-Peer Carpooling & Expense Splitter

Contents

1. **Requirements Table**
   5 Functional + 2 Nonfunctional Requirements
2. **UML Use-Case Diagram**
   Actors, use cases, «include» / «extend» relationships
3. **Use-Case Flow Specification**
   UC-03 – Book a Ride: main flow + alternate flow

Actors: Rider Commuter · Driver Host · Payment Gateway (external)

1

Requirements Table

Functional & Nonfunctional Requirements

PES University – Dept. of CSE

Roopesh · SRN: PES1UG24AM393

Lab 1: Requirements Engineering – Requirements Table

Problem Statement #26: Peer-to-Peer Carpooling & Expense Splitter

Actors: Rider Commuter, Driver Host, Payment Gateway (external)

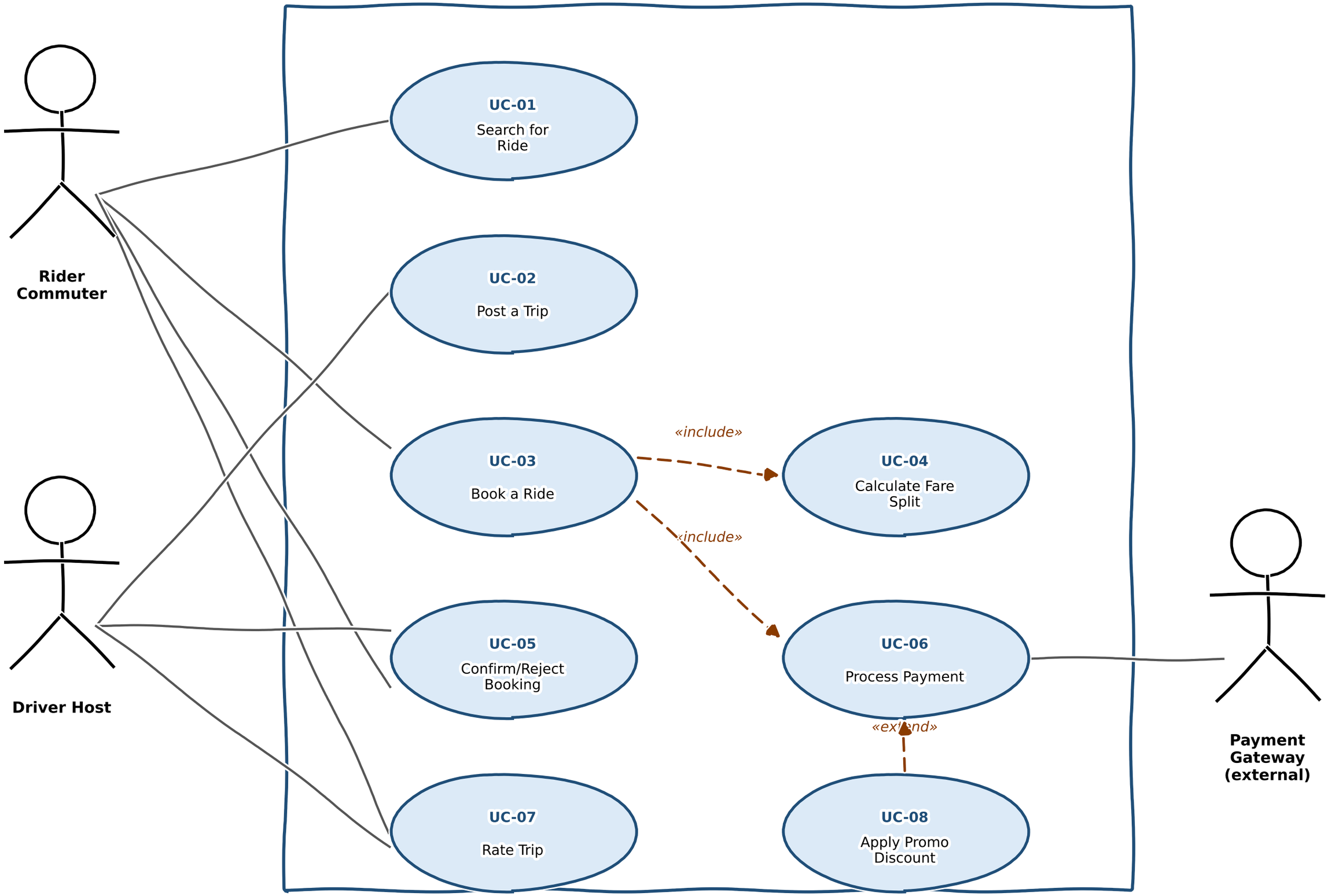
Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall calculate route compatibility between driver and passenger commute paths, permitting matches only within a 1.5 km detour tolerance.	High	Pass – a route match is confirmed and the fuel cost split is calculated. Fail – a match is accepted even though the detour is over 1.5 km.	If the detour is too long the driver loses time for basically no reason, so this keeps matches actually worth it for them.
FR-002	Functional	The system shall let a Rider Commuter search for rides by entering their origin, destination and a rough departure time.	High	Pass – only rides inside the 1.5 km detour limit and within 15 minutes of the requested time show up, ordered by how close the pickup is. Fail – a ride outside that window still shows up in results.	Without filtering, riders would have to scroll through every trip posted that day just to find one that actually works for them.
FR-003	Functional	The system shall let a Rider Commuter send a booking request to a Driver Host, and let the Driver Host accept or reject it within 10 minutes.	High	Pass – status changes to Confirmed or Rejected and both users get notified within 30 seconds of the decision; if no response comes in 10 minutes the request auto-expires. Fail – a request just sits at Pending past 10 minutes.	Nobody wants to be stuck waiting on a maybe. A hard timeout means the rider can always move on to another ride.
FR-004	Functional	The system shall work out each confirmed rider's share of the fuel cost based on distance travelled and how many people are in the car, and show it before the booking is locked in.	Medium	Pass – fare shown = total fuel cost ÷ number of confirmed riders, and it updates the moment someone joins or drops out before departure. Fail – the fare stays the same even after the rider count changes.	This is basically the whole point of the app – splitting fuel cost automatically – so it has to stay correct in real time, not just at the moment of booking.
FR-005	Functional	The system shall let a Rider Commuter and Driver Host rate each other (1–5 stars) and add an optional comment once a trip is marked complete.	Medium	Pass – rating + comment gets saved against that specific trip ID and updates the other person's average rating within 24 hours. Fail – a rating goes through without being tied to an actual completed trip.	You're getting into a stranger's car (or letting one into yours), so some kind of trust signal from past trips matters a lot here.
NFR-001	Nonfunctional	User ratings, driving license verification documents, and travel histories must be stored securely with encrypted backups.	High	Pass – benchmarking tests confirm target latency and security standards under simulated peak load.	This is sensitive, identity-linked data, and the whole safety model of the app depends on people trusting it's handled properly.
NFR-002	Nonfunctional	The system shall show route-match results to a Rider Commuter within 3 seconds when searching within a 10 km radius.	Medium	Pass – 95th percentile response time stays under 3 seconds with 500 concurrent searches simulated. Fail – more than 5% of requests take longer than 3 seconds.	People check this on their way out the door for their commute – if it's slow they'll just give up and drive themselves.

2

UML Use-Case Diagram

Actors, Use Cases & Relationships

Carpooling & Expense Splitter System



Association
«include» / «extend» (dependency)

Roopesh · SRN: PES1UG24AM393

3

Use-Case Flow Specification

UC-03 – Book a Ride

Roopesh · SRN: PES1UG24AM393

Use-Case Flow Specification

Problem Statement #26: Peer-to-Peer Carpooling & Expense Splitter

Use Case: UC-03 – Book a Ride
Primary Actor: Rider Commuter
Secondary Actors: Driver Host, Payment Gateway

Preconditions

- Rider Commuter is logged in and has a verified profile.
- At least one ride within the 1.5 km detour tolerance is already showing in the rider's search results (comes from UC-01 Search for Ride).
- The Driver Host has posted that trip and still has a free seat (UC-02 Post a Trip).

Postconditions

- Success – the booking is confirmed, the fare split is worked out, and both sides get notified.
- Failure – nothing is booked and the seat stays open for someone else.

Main Success Scenario

1. Rider Commuter picks a ride from the results and taps “Request to Book.”
2. System works out the rider's share of the fuel cost using trip distance and the current number of confirmed riders («include» Calculate Fare Split) and shows it to them before anything is finalised.
3. Rider Commuter reviews the fare and confirms the request.
4. System forwards the request and fare estimate to the Driver Host, and starts a 10-minute countdown for a response.
5. Driver Host looks over the request and accepts it.
6. System recalculates the fare split now that the rider count is final, and runs payment authorisation («include» Process Payment).
7. Booking is confirmed – both the Rider Commuter and Driver Host are notified and the trip shows up on both of their schedules.
8. Use case ends successfully.

Alternate Flow – A1: Driver Host rejects the request, or it times out

5a. Instead of accepting, the Driver Host declines – or just doesn't respond within the 10-minute window.

- 5a1. System cancels the pending booking and frees up the seat again.
- 5a2. Rider Commuter is notified that the request didn't go through.
- 5a3. Rider Commuter is sent back to their search results (UC-01) to try a different ride.
- Use case ends in failure – the rider isn't charged anything.